

Agentic HPC 初期最適化における客観的 Evidence と その LLM 解釈の増分効果

樹神宇徳

名古屋大学大学院情報学研究科
愛知県名古屋市

要旨—大規模言語モデル (LLM) によるベストファースト木探索 (BFTS) では、親の実行可能 **workspace** に加えて、外部評価器の客観的 **Evidence** と親 LLM の自己報告を子へ継承できる。本稿は、**Workspace only** (明示的な親 **report** なし)、**Evidence only**、**Evidence+Self-Report** という 3 種類の **bundled policy** を比較する。**Self-Report** は親モデルによる作業記述、懸念、次手提案であり、親自身のツール出力や自己テストは参照し得るが、外部評価器の結果を入力として生成しない。**gemm**, **spmm**, **stencil** について、同じ **seed** を持つ 3 方針の別実行を 1 **block** とする 30 **matched seed block**、計 270 **run** を A64FX 上で実施し、選択候補を別 **seed**・各ケース 15 反復で再測定する。

本稿はあわせて、この比較を成立させるための評価系を示す。予備実験の監査により、分母が **naive** 実装であること、作業集合が最終段キャッシュに収まること、計上時間に **wall clamp** を掛けること、**OpenMP thread team** を計測窓の内側で生成することが、いずれも条件差と同程度以上の系統差を生むことが判明した。そこで分母を **competent** な凍結参照に置き換え、キャッシュに収まらない問題サイズを選び、計上時間を候補の内部タイマに一本化し、**team** 生成を計測窓の外へ出す。偽装は、候補のカーネル時間を測っていない **wall** 下界ではなく、**export** 記号・初期化フックの制限と依存記号の検査によって構造的に防ぐ。

本設計が同定するのは、自然言語を完全除去した介入ではなく、評価後の **Self-Report** を明示的に常時追加する **policy bundle** の **end-to-end** 効果である。意味内容、文脈長、品質、**workspace** 内の自然言語は独立には同定できない。

Index Terms—Agentic AI, high-performance computing, code optimization, self-report, tree search, experimental evaluation

I. はじめに

高性能計算 (High Performance Computing; HPC) に対する LLM コードエージェントは、編集・ビルド・計測を反復しながら候補を探索する。BFTS では高得点ノードを展開し、子は親のコードと **workspace** を引き継ぐ。ここへ評価結果を追加すれば、親コードの正当性、ケース別性能、失敗理由を明示できる。さらに親 LLM の作業記述、懸念、次手提案を渡せば、探索を方向付けられる可能性がある。一方、**workspace** 自体もコメントや追加ファイルとして自然言語を保持し得る。本稿では、実装上 “Reflection” と呼ばれていた 3 フィールドを、その生成過程に即して評価後の親 **Self-Report** と呼ぶ。これは評価器が当該ノードを採点した後に、その判定を入力として生成される。

本研究の中心課題は、実行可能な親 **workspace** に明示的な客観 **Evidence** を追加する **end-to-end** 効果と、

さらにその **Evidence** に対する LLM の解釈をすべての **handoff** に追加する増分効果である。**Self-Report** は評価器の判定を見た後に生成するため、**Evidence+Self-Report** と **Evidence only** は同一の客観的証拠を持ち、差は解釈の有無だけになる。すなわち **Evidence+Self-Report**—**Evidence only** は「客観的証拠の上に LLM の解釈が何を足すか」を測る。

当初の実装は **Self-Report** を評価前に生成しており、その場合の対比は「証拠+判定を見ずに書いた自己記述」対「証拠のみ」となる。これは分離としても不完全であった。エージェントの自己テストは採点対象と同じ形状を測り、課題文はその値を参照値で割ればスコアの良い近似になると明示しているため、自己記述は判定を知らないわけでもなかった。評価後に移すことで、両腕が同じ証拠を持つ設計になる。

代償として、解釈が判定の数値をそのまま繰り返せば **Evidence+Self-Report** は **Evidence only** の言い換えになりうる。そこで生成 **prompt** は測定値の再掲を禁じ、結果がコードについて何を含意するかのみを求める。提案が実際に子コードへ到達したかは、提案を受けていない条件での基底率と対比して事後に測る (表 VI)。

本稿の貢献は次の通り。

- 1) **Evidence** と、その **Evidence** に対する LLM 解釈を分離し、後者の常時追加だけを変える主要対比を定義する。
- 2) **handoff** 方針を **run** 単位で固定した 270 **run** の対応実験により、**end-to-end** の探索性能を評価する。
- 3) 予備実験の監査から、分母・問題サイズ・計上時間・**thread team** 位置が条件差と同程度以上の系統差を生むことを実測で示し、これらを比較可能性の前提条件として評価系に組み込む。
- 4) **token**、実行時間、**Self-Report** の欠損・品質、**Workspace only** の自然言語漏洩、絶対性能を事後監査し、主介入の同定範囲を明示する。

II. 関連研究

最も近い先行研究 AIDE [1] は ML エンジニアリングをコード空間の木探索とし、各ノードを完全なスク립トと要約 $\Sigma(T)$ で表すが、要約中の客観的証拠と LLM 解釈を分離していない。CodeTree [2]、LATS [8]、GIF-MCTS [3]、Tree of Thoughts [7] は **critic**、**scoring**、探索方策を扱うが、**handoff** の情報成分を比較しない。

Reflexion [6] は言語的反省を記憶するのに対し、本研究は客観的証拠と、その証拠に対する親の解釈を分離する。すなわち自己反省の有用性全般ではなく、それをすべての handoff に追加する bundled policy が Evidence only を上回るかを問う。KernelBench [4] は GPU カーネルの逐次改良であり、分岐木上の親子継承を対象としない。本研究は、Evidence と Self-Report を分離し、run 単位で統制したうえで複数の HPC カーネルを同じ原則で評価する。

LLM 以前から、ATLAS [9]、FFTW [10]、OpenTuner [11] は実機測定に基づく自動チューニングを行い、TVM [12] は学習したコストモデルで低水準プログラム探索を高速化する。Kokkos [13] は異種 many-core 環境での performance portability を扱う。これらは、有効な最適化が課題とアーキテクチャに依存することを示す。本研究は既存 autotuner の代替ではなく、LLM 探索で実測情報を試行間にどう伝えるかを対象とする。HPC 向け LLM 評価 [14] と MARCO [15] は、性能 feedback の可能性と同時に、正当性・並列化・アーキテクチャ理解の失敗を報告する。本研究は model、tools、評価器、計算機、探索予算を固定し、親子間情報の性質だけを ablation する。

近年は、VibeCodeHPC [16] が複数 agent による HPC 反復 tuning、AutoPass [17] が compiler artifact と runtime evidence に基づく性能調整、compiler-LLM cooperation [18] が compiler 構成要素を tool とする最適化を扱う。PerfCodeBench [19] は baseline、参照最適化、実行可能な正当性検査を備えた system-level 評価を提案し、実機性能を reward とする online RL [20] も報告されている。これらが feedback を利用した性能向上を問うのに対し、本研究は feedback 内の機械生成 Evidence と評価後 Self-Report という明示的 channel を ablation する。

III. 対象システム

a) 構成: 実験システムは、ベストファースト木探索 (BFTS)、各 node 内の ReAct [5] ループ、Model Context Protocol (MCP) で公開するコード操作ツール、固定した非 LLM 評価器からなる。入力是最適化課題と初期 workspace であり、アイデア生成、文献調査、論文生成は実行しない。

b) run、node、workspace: 1 run は、1 課題、1 乱数 seed、1 handoff 条件で別個に実行する BFTS 木を指す。node は木中の 1 回の最適化試行であり、各 node は専用の workspace を持つ。workspace は候補 C ソース、候補用コンパイルフラグ、Makefile、自己テスト用ファイルからなるディレクトリである。各課題の root workspace には、正しい naive 実装と自己テスト用足場を配置する。子 node では、親の候補ソースとビルド設定を複製し、親候補を初期状態として次の最適化を行う。したがって Workspace only は、コメントを除去した純粋な code token ではなく、この実行可能 workspace と「明示的 parent-report message なし」を意味する。

c) 1 node の実行: 図 1 に処理順を示す。まず実験制御器は、課題記述、利用可能なツールのスキーマ、条件に応じた親 handoff を LLM へ与える。LLM は ReAct 形式で workspace を読み書きし、ビルドと自己テストを

反復する。終了後、LLM から独立した非 LLM 評価器が候補だけを別途コンパイルし、正当性と性能を測る。実験制御器は評価値を node のスコアとして保存し、完了 node を展開候補集合 (frontier) に加える。空き探索枠が生じると、BFTS は frontier 中で最高スコアの親を選び、その workspace と、実験条件で許された handoff から子生成する。

d) エージェントに公開するツール: ツールは MCP サーバから提供し、全課題・全条件で表 I の 4 個に固定する。予備実験で公開していた describe_environment、run_code、emit_results は公開しない。環境カタログはプロンプトへ前置するため describe_environment は冗長であり、run_code は run_bash と重複する実行経路であり、emit_results は採点に一切寄与しない channel に予備実験で 2,222 step を費やしていた。各呼出しは node workspace を作業ディレクトリとする。

実装ではこの許可リストを条件間で固定し、ツール名、説明、JSON 入力スキーマと提示順も変えない。メモリ検索、評価器呼出し、評価契約生成、ジョブ投入、論文生成の各ツールは LLM に公開しない。特に、採点用ハーネスは MCP ツールではなく、ReAct ループ終了後に実験制御器が呼び出す。

e) 環境情報の経路: エージェントへ渡す環境カタログについて、予備実験には 2 つの欠落があった。第 1 に、本サイトの module tree は 2 段構成であり、素の module avail は entry module のみを列挙する。この平坦な一覧にはコンパイラが 1 つも現れず、ベンダコンパイラは entry module を load して初めて可視になる。したがって予備実験のエージェントはその存在を知り得なかった。本条件では entry module を部分シェル内で load して差分のみを報告する (実際に load すると、記述対象であるノードの構成を変えてしまう)。第 2 に、本機は lscpu・sysfs・getconf のいずれもキャッシュ容量を報告せず (sysfs は L2 を 48 コア共有と誤って報告する)、ブロッキング因子をキャッシュに合わせるという操作が原理的に不可能であった。本条件ではポインタチェイスにより階層を実測し (本機で line 256 B、L1d 64 KiB、段差 4096 KiB、所要 0.66 s)、その結果をカタログに含める。機種固有の値をコードへ埋め込まないため、他機でも正しい値が得られる。最後の段差 4096 KiB は公称の CMG あたり L2 容量 8 MiB と一致しないが、これは実測が単スレッドが実際に依存できる容量を返すためである。実測した遅延は 1024 KiB までの 28 ns から 2048 KiB で 31 ns、4096 KiB で 46 ns と漸増し、公称容量ちょうどの 8192 KiB では 106 ns と主記憶並みに達する。ブロッキング因子を合わせるべき値は後者ではなく前者であり、本稿の評価ケースはいずれも両者を大きく超える。

f) 記録と情報経路: 各 node の終了時に、実験制御器は node_report.json と full trace を保存する。前者は node ID、評価状態、ケース別結果、変更ファイル、LLM の自己記述を生成元別に保持する。さらに measurement_audit に各反復の入力 seed、候補・baseline の内部時間と wall time、wall 下界とその比 (v2 では棄却に用いず診断値として記録する)、残差、採



図 1. 通常 node の時系列。Self-Report は外部評価前に生成され、評価器は自己申告を採点に用いない。

表 I
LLM エージェントが利用できるツール。

ツール	操作	本実験での位置付け
write_code	指定したテキストをソースファイルへ書く	新規ファイルの作成に用いる。コンパイルは行わない。
edit_code	既存ファイル内の厳密一致する断片を置換する	既存候補の部分更新に用いる。old_string が 0 回または複数回一致する場合は失敗させ、曖昧な編集が別の箇所へ着弾しないようにする。
read_file	テキストファイルを範囲指定して読む	候補、Makefile、自己テスト結果を観察する。
run_bash	shell コマンドを実行し、標準出力、標準エラー、終了値を返す	コンパイル、リンク、実行、自己テストに用いる。タイムアウト既定 600s。

否、実効・拒否コンパイルフラグを保存する。full trace は ReAct の全メッセージ、ツール入出力、および step 上限到達後に Self-Report を補完する別 LLM 呼出しの prompt と応答を保存する。これらは監査用であり、許可した Evidence 以外は子へ渡さない。

IV. 実験設計

a) 探索と handoff 条件: 各 node のエージェントは最大 20 ReAct step で上記ツールを用い、候補コードを作る。予備実験では 25 step としていたが、node が自身の最良結果に到達した step は p50=3, p90=13, p95=15, p99=20 であり、上限を 25 から 40 へ引き上げても改善した node は線形外挿の 101 ではなく約 28 にとどまった。あわせて、作業ディレクトリの一覧、継承した候補ソース全文、現行コンパイルフラグ、および実測した環境カタログをプロンプトへ前置する。予備実験ではこれらの再取得に全 step の 27.5% (約 12,749 step) を費やしており、いずれもエージェント自身のツールで到達できる情報であるため、前置は新たな情報を与えず往復のみを削る。固定した非 LLM 評価器が全ノードを採点し、BFTS は最高得点のフロンティアノードを展開する (本実験では $N = 10$ ノード/run)。これは LLM が最適化する root 1 試行と子 9 試行の合計であり、未変更 root の後に 10 個の子を作る設定ではない。子は常に親 workspace を複製し、追加文脈だけを次の 3 条件で変える (図 2)。

- 1) **Workspace only:** workspace のみで、明示的 parent-report message はない。
- 2) **Evidence only:** workspace と、評価器・ハーネスが確認した客観的証拠。
- 3) **Evidence+Self-Report:** Evidence only に、評価後の親 Self-Report を追加。

Evidence+Self-Report では利用可能な全親子 handoff に Self-Report を注入するため、失敗時だけの選択方針ではなく always-on policy bundle を評価する。以下、完全な 3 方針を W, E, S と略す。1 run には 1 方針だけを適用し、探索木は共有しない。同一 seed の 3 run は matched

block だが、親 workspace、prompt、model sample、探索軌跡を共有しない。推定対象は固定親からの局所差ではなく、handoff 方針を通した end-to-end の run 差であり、主要対比は E-W と S-E である。

BFTS は、外部評価結果を持つ frontier node を優先し、その中で非正規化の幾何平均 speedup が最大の node を選ぶ。同点なら現在の frontier で先に現れる node を選び、適格な限り同じ親を再展開できる。子が親を厳密に上回れば親を retire する。無変更子は sterile として測定を保持するが、再展開、親 retire、最終選択から除く。測定 invalid node は score 0 で記録され、高得点の有効 node がなければ debug 用の親になり得る。最終候補は有効かつ非 sterile の最大値とし、完全同点では辞書順に並べた report path の先頭を採る。1 node の再展開上限は 999 で、10-node run では到達しない。

workspace の複製では、候補ソース、ビルド設定、自己テスト足場を継承する一方、results.json、評価結果、標準出力・標準エラー、生ログ、node_report.json、環境プローブ結果を除く。さらに、過去 node の要約を検索・注入する cross-node memory と、BFTS の子生成プロンプトを作るプランナが親レポートを付与する既定経路を、全条件で無効化する。これにより Workspace only に別経路から親の測定値や説明が流入することを防ぐ。ただし、親 LLM がソースコメントまたは任意の非除外ファイルへ書いた文章は、コード成果物の一部として残りうる。

b) Evidence と Self-Report の生成: 表 II に子プロンプトへ出力するフィールドを示す。Evidence は、制御器、評価器、ハーネス、ファイル hash 比較が機械的に確認した 7 フィールドだけから生成する。evaluation_cases はケースごとの valid と measurements を持ち、測定名は各ハーネスが定義する。known_failures は測定無効時の評価器理由から機械的に導出し、LLM の懸念を参照しない。反復ごとの生時間とコンパイルフラグを持つ measurement_audit は監査専用であり、Evidence へ

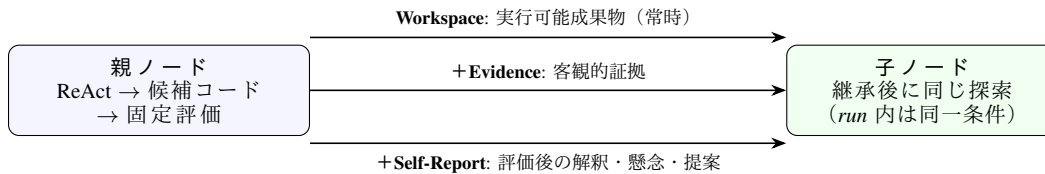


図 2. 親子間 handoff. 各 run は 3 条件のいずれかに固定される。

含めない。

Self-Report は、親 LLM が node 終了時に記録した作業解釈、残る懸念、次の最適化案から成る。両 Evidence 条件は同じ見出しとレンダリング関数を使う。同一 report を入力したときの客観部分は byte 単位で同一であり、Evidence+Self-Report はその末尾へ Self-Report だけを追記する。実際の条件間では探索木を別個に実行するため値自体は異なる。handoff は子の最初の LLM 呼出しに独立メッセージとして注入する。Workspace only ではこのメッセージを生成せず、raw report、監査記録、full trace は全条件で注入しない。監査可能性のため Self-Report 記録自体は全条件で生成する。通常は親の finish JSON として外部評価前に生成され、run_bash 等の自己テスト出力は参照し得るが、後続の外部評価値は参照できない。20 step 到達時は ReAct 終了後に tool なしの補完呼出しを 1 回行う。この呼出しは制御器側評価の後になる path もあるが、prompt は評価前 ReAct trace の末尾だけであり、外部評価結果を含まない。補完呼出しは 20 step の外側だが、token、call 数、wall time に含める。いずれの Self-Report も同一 node の行動 loop へ戻さない。したがって比較するのは評価後 Self-Report の注入効果であり、外部 Evidence を読んだ解釈や生成省略による運用コストではない。

値が存在しないフィールドは出力しない。Workspace only には表 II のフィールドを一つも渡さず、出力・ログ・レポートを除外した親 workspace だけを渡す。

c) 課題と採点: 3 課題はいずれも、正当性を満たした候補の参照実装比 speedup を最大化する (表 III)。参照実装は naive ではなく、同じ契約 (plain C、OpenMP、コンパイルフラグのみ、intrinsics・アセンブリ・外部ライブラリは不可) で書かれた competent な実装である。分母を naive に取ると、スコアは「候補が何もしない状態からどれだけ離れたか」を測ることになり、最適化の質を測らない。

ここで competent の水準は、実測で定義する。予備実験の corpus では gemm の上位 1 割がブロッキング定数を 1 つも持たず、教科書どおりの ijk ループにベクトル化フラグを加えたものが最良であった。したがって参照は少なくともこの実装に負けてはならない。この検査を 3 課題すべてに対し、予備実験でエージェントが実際に生成した最高スコア候補を用いて行った。候補ソースはそのコンパイルフラグごと corpus から取り出し、採点経路で分母と比較した。参照比は spmm が 6 ファミリで 0.613–0.871、stencil が 0.771/0.783/0.812 であり、参照がそれぞれ 1.15–1.63 倍、1.23–1.30 倍速い。

gemm の参照比は 0.777/0.861/0.823 であり、3 形状すべてで参照が 1.16–1.29 倍速い。

d) 採点形状は分母ではなく分散で決まった: この gemm の第 3 形状は当初 $500 \times 500 \times 2000$ であり、ここでは参照が ijk に 1.313 倍劣っていた。ブロッキングが並列反復数を MR で割るため、 $n = 500$ ではタイルが 125 個しかなく 48 スレッドを満たせない。参照が持つ行並列経路へ分岐させると 1.021 まで回復するが、独立に 10 回採点したときの計上時間のばらつきが 0.76 % から 11.87 % へ、反復数の根拠である band 指標が 0.46 % から 2.27 % へ悪化した。レジスタブロッキングを保ったまま NC を下げて並列単位を増やす案は速度・安定性の両方でさらに悪く (NC = 256 で 5.565 ms/9.28 %、128 で 5.722 ms/11.33 %)、この形状で両立する分解は見つからなかった。

原因はカーネルではなく形状であった。 $n = 500$ は 48 スレッドに対し 10.4 行/スレッドであり、候補が実際に書く ijk を含めてあらゆる行並列実装がここでは不安定になる。すなわち分母を安定側に固定しても、候補側のスコアが揺れる問題は残る。 n を上げるだけでは足りず ($1000 \times 600 \times 2000$ で ijk はなお 4.34 %)、 $1500 \times 600 \times 1500$ で解決した。同形状では参照が 7.341 ms/0.18 %、ijk が 8.871 ms/3.37 % であり、安定な経路が同時に速い経路になる。長い縮約という regime ($p/m = 2.5$) と、作業集合が最終段キャッシュを超えるという制約は保たれている。

したがって採点形状の選択は、問題規模やキャッシュ収まりだけでなく、候補が書くであろう実装がその形状で安定に測れるかによっても決まる。この基準は本稿の設計から事後的に得られたものである。

e) 問題サイズの選択: gemm の作業集合は、予備実験で用いた 512^3 では 6.0 MiB であり CMG あたり L2 (8 MiB) に収まっていた。この条件では、独立に書いたレジスタブロッキング実装と、ブロッキングだけを外した実装との比が 1.046 倍にとどまる。予備実験の測定バンド (同一対象を独立に 2 回採点したときの $|\Delta|$ の p95) は探索が用いた反復数で 4.6 % であり、この 4.6 % の利得はちょうど分解限界に等しい。すなわち探索はブロッキングの効果を分解できなかった。作業集合が L2 を離れる本条件では同じ比が 1.289 倍となる。SVE を有効にした対でも同様に、 512^3 の 0.988 倍に対し本条件では 1.229 倍である。すなわちブロッキングという最適化を目的関数が識別できるかどうかは、問題サイズによって決まる。 m を 32 の倍数としないのは、 B の列ストライドが L1 のセットにエイリアスしうるためであるが、本機では効果を確認できなかった。 m だけを変えた対 (2048 対 2040、1024 対 1016) を、アクセスパターンの異なる 3 実装 (凍結参照、ブロッキングを外した

表 II
EVIDENCE と SELF-REPORT の子向けフィールド。上段 7 フィールドは E/S に共通し、下段 3 フィールドは S だけに追加する。

子向けフィールド	内容と生成元
Evidence only / Evidence+Self-Report 共通	
status	制御器が記録した ReAct 実行状態。候補の正当性判定とは区別する。
measurement_valid	評価器が保存する測定有効フラグ。ビルド・実行・全評価ケースを有効と判定したか。
evaluation_cases	評価ケースごとの valid と、ハーネス定義の客観測定値 measurements。共通スキーマは問題固有の測定名を規定しない。
evaluator_reason	固定評価器が返した採択・棄却理由。
key_metrics	metrics から転記する評価値。幾何平均・ケース別 speedup 等を含む。
changed_files	raw files_changed から転記する、hash 比較済みの追加・変更・削除ファイル。
known_failures	測定無効時の evaluator_reason から機械的に導出。LLM の懸念は含めない。
Evidence+Self-Report のみ：評価後の親 Self-Report	
reflection_summary	外部評価値を入力しない、親 LLM による実施内容とボトルネック解釈の要約。
concerns	親 LLM が自己評価で挙げた未解決の懸念。
next_steps	親 LLM が提案した次の最適化案。

表 III
主実験の課題。評価ケースの大きさは、いずれも作業集合が CMG あたり L2 (8 MiB) を超えるよう選んだ。

課題	評価ケースと探索中の反復数	主な最適化特性
gemm	(1000, 1000, 1000)、(2000, 500, 500)、(1500, 600, 1500) ; 各 15 反復。作業集合 22.9/17.2/30.9 MiB。m は 32 の倍数とせず、n はスレッド数の 20 倍以上とする（後者は行並列実装が安定に測れる条件）	規則的・計算集約的；データ再利用、SIMD、並列化
spmm	n = 20000, k = 64 ; 一様・帯・ベキ則・ブロック・対角優位・偏りの 6 疎構造族；各 15 反復	不規則・メモリ依存；CSR アクセス、負荷分散
stencil	(256, 256, 256, 240)、(384, 192, 192, 240)、(192, 192, 384, 240) ; 各 15 反復	局所性、境界処理、並列化、時間ブロッキング、NUMA 配置

ikj、B をストライド参照する naive ijk) について採点経路で実測すると、比は 0.98–1.08 倍に収まり、B をストライド参照する naive でも 2048 で 1.01 倍 (235.9s 対 234.6s) であった。この制約は採点ケースが既に満たしているため保持するが、根拠は実測に基づかない。

stencil の掃引回数 n_t を 240 とするのは、候補が自前で ping-pong バッファを確保する仕様による。268 MB 分の first touch に伴うページフォルトが計測窓の内側に入るため、 $n_t = 30$ では計測窓 78.5 ms のうち 50.6 ms (64%) が掃引以外であった。計測窓は $n_t = 30, 60, 120, 240$ の実測に対し $50.57 + 0.9298 n_t$ [ms] に適合し (最大残差 0.4%)、 $n_t = 240$ ではこの比が 18.5% となる。

評価器は候補と凍結 baseline を別々にコンパイルし、fp64 参照解と要素ごとに照合する。出力要素 z の許容誤差は

$$|z_c - z_r| \leq \begin{cases} c_\epsilon \gamma(p) (|A||B|)_{ij}, & \text{gemm,} \\ c_\epsilon \gamma(\text{nnz}_i) (|A||X|)_{ij}, & \text{spmm,} \\ c_\epsilon \gamma(7) \max(n_t, 1) \max |u_0|, & \text{stencil,} \end{cases}$$

とする。ここで $\gamma(k) = ku/(1 - ku)$ 、 u は fp64 の unit roundoff、 c_ϵ は固定安全係数、 p は縮約長、 nnz_i は第 i 行の非零数である。

各反復で新しい入力を生成し、候補と参照を交互順で対応実行する。奇数反復で一方が 1 回多くなるため、開始順を seed の奇偶で反転し、30 seed 全体では各条件の実行順を釣り合わせる。計上時間は候補プロセスの内部タイマとし、wall clock による clamp は行わない。wall 下界 $t_{c,\text{wall}} - (t_{b,\text{wall}} - t_b)$ は候補のカーネルについて何も測っていない。差し引く $t_{b,\text{wall}} - t_b$ はプロセス起動・問題読込・出力書出しであり、本機では gemm 18.9–21.1 ms、spmm 56.5–91.3 ms、stencil 109.0–127.0 ms を占める。これは凍結参照の wall 全体の gemm で 80–88%、spmm で 81–94%、stencil で 32–34% にあたる。すなわち下界の大半はカーネル外の量で決まり、参照の overhead が候補の wall 全体を上回れば負にもなる。閾値による救済も成立しない。予備実験の構成での実機検証では、閾値 0.50 が本研究の選抜候補自身を 3 反復中 2 反復で棄却し、0.02 でも棄却した。カーネルが実際に速くなるほど (当該構成で 0.8–1.5 ms、下界は約 60 ms) 正直な比は 0.015 程度まで下がるため、速い正直なカーネルと偽装を分ける閾値は存在しない。偽装は代わりに構造的に防ぐ。候補オブジェクトは所定のカーネル記号のみを export でき、

.init_array/.preinit_array/.ctors/.fini_array/.dtors と IFUNC を持つ場合、および fopen/open/atexit/system/fork/ exec*/dlopen

等を参照する場合は無効とする。また OpenMP の `thread team` はタイマ開始前に生成する（候補カーネルは呼ばない）。この 1 点だけを戻した対照を実測すると、凍結参照の内部時間から 48 スレッドの `team` 生成に相当する時間が一律に外れる（`gemm` で 3.06–3.12 ms、`spmm` で 2.86–3.27 ms）。v2 の参照はこの時間が無視できる規模ではなく、割合では `gemm` で 37.5–54.3 %、`spmm` で 14.0–41.9 % を占める。`stencil` ではさらに大きく、計測窓が 214 ms から 437–549 ms へ 2 倍以上に伸びる。しかもこの 2 倍は形状によらず一定ではない。反復ごとに別プロセスで実行しているにもかかわらず、ある形状では最初の 3 反復が 437 ms、以降の 4 反復が 216 ms となり、ノード側に持ち越された状態に依存した（この形状の変動係数は 35 % であり、タイマ前生成では 0.2 % である）。すなわち予備実験は `pin` されていない参照を `pin` された候補と比較しており、本条件の `speedup` は予備実験の値と比較できない。正当性と時間検査を通ったペアだけを採用する。ケース i の速度比 g_i は採用ペアの t_b/t_c の中央値とし、過半数未満なら無効とする。全評価ケースが有効な場合だけ

$$\text{valid} = \text{compiles} \wedge \bigwedge_i (\text{valid}_i \wedge g_i > 0 \wedge \text{finite}(g_i)),$$

$$g = \begin{cases} \exp\left(k^{-1} \sum_{i=1}^k \log g_i\right), & \text{valid,} \\ 0, & \text{otherwise} \end{cases}$$

と採点する。無効ケースを部分点として幾何平均へ混ぜない。 g は探索中の BFTS 順位付けと候補選択にそのまま用いる。親との hash 差分がない `no-op` 子は、測定値と `valid` 判定を客観的記録として保持する一方、探索適格性を示す `_sterile` を付けて再展開、親の `retire`、最終候補選択から除外する。探索深さ上限は 10 に固定し、`root` を含む 10 `node` を各 `run` に要求する。`readable` な `node_report.json` が 10 件に満たない `run` は、条件依存の計算量差として扱わず `infrastructure_error` とする。探索終了後、最大 g の有効な非 `no-op` 候補を別 `seed` (`run seed +106`)、各ケース 15 反復で再測定し、この再測定値を `run` の主要アウトカムとする。再測定で無効なら 0 とする。

f) 測定防御と最適化フラグ: 採点ハーネスと `baseline` は `node workspace` 外に置き、`manifest` の `sha256` と一致しない場合は `infrastructure_error` として `run` を停止する。評価には `workspace` の候補実装だけを取り込み、各反復を新しい入力・プロセスで実行する。出力は NaN で初期化し、参照解との不一致、短い出力、異常終了、および内部時間が非有限または非正である反復を `fail-closed` で棄却する。`wall` 下界との比は棄却に用いず、診断値としてのみ記録する。1 プロセスの実行上限は 120 秒に固定し、候補の超過は `candidate_invalid`、凍結 `baseline` の超過は `infrastructure_error` とする。いずれか1ケースで候補が無効なら、全ケース必須の採点は既に無効なので残りのケースを実行しない。課題契約は外部の既製カーネルへの処理委譲を禁じ、リンク・`include` 用フラグも拒否する。

候補はコードとコンパイル最適化フラグを探索できるが、`baseline` は既定フラグに固定する。`allowlist` は最

表 IV
反復数 15 における測定 `BAND` (同一ビルドを 12 回採点)。目標は予備実験の兄弟間差中央値の 1/3。括弧内は予備実験の測定系で得ていた値。

課題	band	目標	超過	1 採点
<code>gemm</code>	0.251 % (0.274)	0.237 %	1.06 倍	13.1 s
<code>spmm</code>	0.171 % (0.358)	0.170 %	1.005 倍	56.8 s
<code>stencil</code>	0.149 % (2.633)	0.107 %	1.40 倍	97.7 s

適化・チューニング用フラグだけを許可し、外部リンク、`include path`、出力先、ツールチェーンの変更を拒否する。実効・拒否フラグ、評価設定、ハーネスの `sha256` を記録する。

g) 反復数と、その分解限界: 反復数 15 は、同一対象を独立に採点したときの再現限界から選ぶ。凍結参照を候補として 12 回採点し、 $|\text{score} - \text{median}|/\text{median}$ の `p95` を `band` と定義した結果を表 IV に示す。目標は予備実験の兄弟間差の中央値の 1/3 とし、課題ごとに異なる。

`stencil` の `band` は予備実験の測定系で 2.633 %、すなわち目標の 24.6 倍であった。本稿の測定系では 0.149 % である。同じ `band` ツール・同じ統計量（12 試行の `p95`、最悪 1 点のみ除外）で測っており、外れ値そのものが出ない。12 試行は 0.99841–1.00111 に収まり、予備実験で見えていた 1.0272 に相当する試行はない。

何がぶれていたか: 機構は特定した。ぶれるのは掃引ではなく固定項である。採点対象バイナリを直接反復すると測定は

$$t = 50.566 + 0.9299 n_t \quad [\text{ms}] \quad (1)$$

に 0.03 % で従い ($n_t = 1, 30, 120, 240$ 、各 20 プロセス)、掃引 1 回あたりの費用はこの全域で再現する。一方、掃引が測定 1.8 % しかない $n_t = 1$ では 100 プロセスの範囲が 33.9–52.0 ms と 34 % に広がる。固定項は、候補が自身の 268 MB を確保し `first touch` する費用が計測窓の内側にあることそのものであり、3 課題で `stencil` だけが持つ構造である。

その固定項を跳ね上げるのはページキャッシュの冷えである。14 回連続採点（反復数 15、3 形状）では、中央値を 5 % 超えた反復 31 件がすべて最初の 2 採点に集中し、3 回目以降は 0 件であった（最大 2086 ms、通常 214 ms）。参照解は 1 形状 1 反復あたり 134 MB の `npz` として共有ファイルシステムに置かれ、合計 5.1 GB を読み込むまで、親の入出力が子の計測窓を止める。実際、採点前にこの 5.1 GB を読んでおく（2 s）だけで最初の 2 採点の外れ値は消え、採点 1 が 2.1–5.3 %、採点 2 が 1.3–1.8 % に収まる。

しかしこれは `score` をぶらさない。冷えたキャッシュは候補と参照を同じ反復の中で同じだけ遅らせるため、比では相殺する。`band` 測定の試行 1 は 105.0 s と他の 97 s より長いのに `score` は 1.00111 であり、14 回連続採点でも外れ値を含んだまま速度比は 0.9986–1.0024、`band` は 0.218 % であった。予備実験で `band` が 2.633 % だったのは、`credited` 時間が壁時計由来であった測定系では、この共通項が相殺しなかったためと解釈できる。本稿の

測定系は `credited` をカーネル内部タイマとしており、相殺が成立する。

したがって `band` は 3 課題とも目標の 1.4 倍以内に入る。ただし達成したとは主張しない — `gemm` 1.06 倍、`spmm` 1.005 倍、`stencil` 1.40 倍はいずれも目標超である。`band` 未満の兄弟選択は分解できていないものとして解析する。主要対比は 30 seed block の対応差に対する検定であって単一兄弟の選択ではないため、この限界は主結論より探索経路の解釈に効く。また目標値は予備実験の兄弟間差から導いており、分母を替えた本条件で同じ差の分布になる保証はない。本条件での兄弟間差は再取得したデータで測り直す。

h) 絶対性能と、到達可能な天井: `speedup` は比であり、それ自体は「妥当なほど性能が出ているか」に答えない。そこで機械の到達可能な演算天井を実測し、参照の絶対性能をその比として報告する。

天井は `SVE intrinsics` (`svmla_f64_x`) で $N_a = 16$ 本の独立な蓄積器を回して測る。各スレッドは I 回の反復で N_a 個の FMA を発行し、1 FMA は 1 レーンあたり 2 flop、レーン数 L は実行時に `svcntd()` で取得する (本機で $L = 8$)。 T スレッド、経過 t 秒として

$$\text{GF/s} = \frac{2 N_a L I T}{t} \times 10^{-9}$$

とする。ループ内はレジスタのみで、メモリ参照も分岐もない。 $I = 2 \times 10^7$ 、 $T = 48$ 、 $t = 0.097\text{s}$ で 2534 GF/s を得た。測定コードは `workspace/tools/measure_fma_ceiling.c` として成果物に含める。`clang` もしくは `fcc -Nclang` でビルドする必要がある (gcc 8.5 には `arm_sve.h` が無い)。

この値には 2 つの検査を課す。第 1 に `objdump` で `SVE` の FMA が実際に発行されていること。ここで `clang` が出すのは `fmla` ではなく `fmad` (例: `fmad z23.d, p0/m, z24.d, z25.d`) であり、`fmla` だけを検索すると 0 命令と誤判定する。第 2 に反復数 I を倍にしても GF/s が変わらないこと (2532.7 に対し 2536.2、0.14%)。前者は自動ベクトル化に頼った版が `SVE` の FMA を 1 つも発行せず、後者はデッドコード除去が起きうるためである。実際、この 2 つを課す前の 6 版はいずれも参照より低い天井を報告しており、天井が下界を割るという矛盾で誤りを検出できた。同様に周波数の測定コードも `workspace/tools/measure_clock.c` として含める。

動作周波数も実測する。本機には `perf` バイナリが無く、`cpu0` 配下に `cpufreq` ディレクトリも無く、`/proc/cpuinfo` は `BogoMIPS 200` しか報告しない。しかし PMU 自体は `/sys/bus/event_source/devices/armv8_pmu/v3_0` として公開され、`perf_event_paranoid` は 0 であるため、`perf_event_open(2)` を直接用いれば非特権プロセスからサイクルを数えられる。1 コアを固定して `busy` 状態で計数すると 1.9984 GHz、1 秒区間と 3 秒区間で 0.00% の一致であった。欠けていたのは計数機構ではなく計測ツールである。

ここで本稿は当初、計測手段が `perf_event_open` しかないとして記述していた。これは誤りであり、探し方が狭かったことによる。`perf/PAPI/likwid` を `module` 名で探していずれも見つからなかったことから「プロファイラが無い」と結論していたが、`PAPI` は `/usr/bin/papi_avail` として導入されており、Fujitsu 純正の `fapp/fipp` (および `fappxx`, `fippxx`, `hwpfctl`) は、本稿が候補に選択させている `fcc` と同一のディレクトリ `/opt/FJSVstclanga/cp-1.0.30.01/bin/` に存在する。コンパイラをそこから起動しておきながら、同じ場所を見ていなかった。`fapp` (Version 4.2.6) は GFLOPS とその理論ピーク比、メモリスループットとその理論ピーク比、SIMD 命令率、SVE 演算率、IPC を名前付きで直接報告する。本稿が `objdump` と反復試行で確認した「自動ベクトル化が `SVE` を発行していない」という事実は、`fapp` では `SVE operation rate` の 1 行で得られる。

したがって理論ピークは $48 \text{ コア} \times 1.9984 \text{ GHz} \times 8 \text{ レーン} \times 2 \text{ flop} \times 2 \text{ FMA パイプ} = 3070 \text{ GF/s}$ となる。実測天井 2534 GF/s はその 82.6%、すなわち 1 サイクルあたり 1.65 FMA であり、`intrinsics` で書いても 2 FMA/cycle には届かない。凍結参照は `gemm` の 3 形状で 382/388/368 GF/s、理論ピーク比 12.4/12.6/12.0%、実測天井比 15.1/15.3/14.5% である。

参照は契約内 (plain C、OpenMP、コンパイルフラグのみ) で書かれており、`intrinsics` もアセンブリも用いない。したがってこの水準は参照の実装品質だけでなく契約そのものの上限を反映する。実測でも、自動ベクトル化に依存する限り `SVE` の FMA が発行されないことが 2 つのコンパイラで確認されており (gcc は蓄積器を回避し、`clang` は `SVE FMA` を 0 命令)、候補が到達しうる性能はコンパイラの自動ベクトル化能力に律速される。予備実験に対する「生成コードの性能が低すぎる」という指摘は、探索の失敗としてではなく、まずこの構造として説明される。

i) コンパイラの選択と固定: ツールチェーンの選択は探索空間に含める。実機チューニングでは当然の軸であり、本機での効果は大きい。凍結参照と同一のソースを、採点と同じ反復数 15 で独立に 3 回ずつ測った結果を表 V に示す。効果は課題によって向きも大きさも変わる。gcc に対し `clang` は `gemm` で差がなく (1.0018 対 1.0048、ばらつきはいずれも 1% 未満)、`spmm` では 3% 遅い。ベンダの `fcc` は既定フラグでは `gemm` で 0.217、`spmm` で 0.086 と大きく劣るが、`-Nclang` を加えると `gemm` で 1.484 (6.8 倍改善、gcc をも 48% 上回る)、`spmm` で 0.984 (11.4 倍改善、gcc と互角) となる。

`stencil` では `clang` と `fcc` の測定が成立しなかった。3 回の値が 1.127/2.255/1.128 のようにちょうど 2 倍に分かれ、幅は中央値の 100% に達する。同じノードで gcc は 0.4%、また `gemm` と `spmm` では `clang` も 0.3–0.6% に収まるため、ノード不調でもコンパイラ固有の不安定性でもない。`stencil` は 3 課題のうち候補が自身で 268 MB を確保し `first touch` を計測窓の内側で行う唯一の課題であり、コンパイラが生成する確保・初期化コードの差がページ配置を変えると解釈してい

表 V

ツールチェーン別の参照比（反復数 15、独立 3 回の中央値。括弧内は 3 回の幅／中央値）。STENCIL の CLANG/FCC は測定が成立しなかった（本文参照）。

	gemm	spmm	stencil
gcc 8.5.0	1.005 (0.9%)	1.003 (0.5%)	0.998 (0.4%)
clang 20.1.8	1.002 (0.6%)	0.970 (0.3%)	—
fcc 4.11.1	0.217 (1.8%)	0.086 (0.2%)	—
fcc -Nclang	1.484 (0.8%)	0.984 (2.2%)	—

た。実行順序は原因ではない（候補先行 96 回の中央値 1.1342 に対し参照先行 72 回で 1.1386、差 0.4%）。

この解釈は方向としては正しかったが、制御していない要因があった。本機の Fujitsu 環境にはラージページ確保方針を定める環境変数 `XOS_MMM_L_PAGING_POLICY` があり、静的領域・ヒープ・スタックそれぞれについて `prepage`（開始時に一括確保）と `demand`（初回アクセス時に確保）を選ぶ。`prepage` では並列 *first touch* ループが走る前にページが確保されるため、確保したスレッドの CMG に全て載り、4 CMG 構成での NUMA 配置が成立しない。この変数を読むのは `/opt/FJSVxos/mmm/lib64/libmpg.so.1` であり、`ldd` で確認すると Fujitsu コンパイラでビルドしたバイナリにのみリンクされる。gcc ビルドでは誰も読まないで設定は無効である。

凍結参照を用いて実測すると、gcc ビルドは 3 設定いずれでも $n_t = 1$ で約 50ms、 $n_t = 240$ で約 273ms と区別できないのに対し、fcc -Nclang ビルドは未設定で $n_t = 240$ が 1375ms、`demand:demand:demand` を与えると 232ms となり 5.9 倍動く。固定項も 30.0 から 16.8ms へ 44% 減る。しかも 232ms は gcc ビルドの 273ms を 15% 上回る。

影響は stencil に限られる。同じ設定で fcc -Nclang を候補として採点すると、gemm は未設定 1.1375/1.1481 に対し demand で 1.1359/1.1365、spmm は 0.9746/0.9590 に対し 0.9608/0.9521 で、いずれも測定幅の内側である。3 課題のうち候補が自身の 268MB を確保し *first touch* を計測窓の内側で行うのは stencil だけであり、他の 2 課題は問題データを外から受け取るのでラージページ確保方針が計測窓に入らない。

したがって表 V の gemm と spmm の値は影響を受けない。一方 stencil の測定不成立は、この変数を制御せずに測ったものであり、そのまま信頼できない。ただし観測された「ちょうど 2 倍」の分裂 (1.127/2.255/1.128) は上記の 5.9 倍とは比が異なり、同一の現象と断定はできない。stencil の非既定ツールチェーンは、この変数を固定したうえで測り直す必要がある。候補は `candidate_cc.txt` に gcc/clang/fcc のいずれかを書く。候補プロセスは凍結 driver を含めて選択したコンパイラでビルドする（オブジェクトを混ぜると OpenMP ランタイムが混在し、計測窓の外で生成する thread team がカーネルと別ランタイムになる）。分母は動かさない。参照は常に既定ツールチェーンでビルドし、スコアは 1 つの固定された対象に対する比であ

り続ける。

分母を 2 つ測る：ただしこの分母だけでは、候補が非既定コンパイラを選んだ瞬間にコンパイラ境界を跨いだ比較になる。境界の片側だけに作用するものは、どちらのプログラムも変えずに比を動かす。前述の `XOS_MMM_L_PAGING_POLICY` がその実例で、候補側（Fujitsu ビルド）だけを 5.9 倍動かし、参照側（既定ツールチェーン）には触れない。反復ごとの対測定は時間変動する条件を相殺するが——冷えたページキャッシュが 214ms を 2086ms にしても比では消える——2 つのビルドの間の非対称は相殺できない。

そこで同一の凍結ソースから 2 つ目の分母を作る。候補が選んだのと同じコンパイラ・同じフラグで参照をビルドし、同じ反復の中で測る。得られる比では、片側だけに効く要因が両側に効いて消える。2 つの比は置き換えではなく併記であり、`speedup` は従来どおり `anchor`（既定ツールチェーン）比であり続ける。

- **anchor 比**——コンパイラ選択を含む end-to-end の結果。本稿のスコアはこれである。
- **matched 比**——コードの寄与のみ。同一コードなら 1 になる。
- 両者の商——ツールチェーンが買ったもの。

凍結参照を候補として採点して検証すると、matched 比は 3 課題 × 2 コンパイラで 0.9914–1.0055 に収まる（同一プログラムなので 1 が正しい）。同じ実行の anchor 比は 0.1735–1.0906 と 6.3 倍に散らばり、その全部がツールチェーンとフラグの寄与であって、コードの寄与ではない。従来の測定はこの 2 つを分離していなかった。

この *anchor* 幅そのものが運用設定に依存する。上記は `XOS_MMM_L_PAGING_POLICY` を固定せずに測ったもので、同じ検証を `demand:demand:demand` を与えて再実行すると matched は 0.9966–1.0047 と変わらないのに、anchor は 0.4702–1.1155、すなわち 6.3 倍ではなく 2.37 倍になる。差は fcc の stencil 一点に集中しており、5.9 倍の変数が候補側だけを動かすという前述の観測と一致する。本稿がこの 2 つを条件付きで併記するのは、条件を書かない比率がまさに本節の主張する誤りだからである。matched 比が両条件で 1 の近傍に留まることが、分離が働いているという主張の本体であり、anchor 幅はその対比として読むべき量である。検証は `workspace/tools/check_matched_denominator.py` で再実行でき、分離が壊れると非零で終了する。コストは候補が非既定コンパイラを選んだ反復でのみ実行が 2 回から 3 回に増える。

測定条件そのものを記録し、照合する：2 分母は 1 回の採点の内部にある非対称を扱う。残るのは、採点と採点の間で条件が動く場合である。5.9 倍の変数は運用設定であって実験設計ではないから、管理者が既定を変えれば、月曜のシャードと金曜のシャードは別の条件で測られる。このとき何も失敗しない。全シャードが採点され、全数値がもっともらしく、条件間の対比だけがエージェントと無関係な因子を吸収する。反復ごとの対測定は 1 回の採点の内側で変動する条件を相殺するが、採点の間で変わった条件は相殺しない。

そこでハーネスは、測定に関わる環境変数を採点のたびに記録し、その要約 (sha256) を結果に付す。捕捉

は名前の列挙ではなく接頭辞の走査で行う。手書きの名前リストは誰かが思いついた変数しか記録しないため、実際、本研究のスタディ manifest は 24 変数を名前固定していたが `XOS_MMM_L_PAGING_POLICY` はその中になかった——後から見つかった変数だからである。逆にハーネス側は BLAS 系スレッド変数を捕捉しておらず、2 つの部分的な記録が併存し、どちらも他方を含んでいなかった。部分的な記録が 2 つあるのは 1 つより悪い。どちらも完全に見えるからである。両者は単一の捕捉に統合した。値は記録するが制限はしない。接頭辞の外にある変数は捕捉もされず、無害と分かってもいない——記録はその旨を明示して保存する。

記録は照合しなければ意味を持たない。`workspace/tools/check_environment_drift.py` はスタディ配下の全 digest を数え、複数あれば差分変数を提示して非零で終了する。判定はしない。「その変数が測定に届くか」は事実ではなく判断だからである。記録が 1 件も無い場合も clean とは報告しない——捕捉前の古いスタディと捕捉が壊れたスタディは、外からは区別できない。実機検証では、5.9 倍の変数のみを変えた 2 回の実採点が異なる digest を生み、検査器がその 1 変数を特定した。

フラグ許可リストはベンダの名前空間 `-K*/-N*` を含む。これは形式的な追加ではない。同一の `fcc` が、`gemm` では既定フラグで参照比 0.217、`-Nclang` を加えると 1.484 となり 6.8 倍動く。すなわち `-Nclang` を拒否していた当初の許可リストは、`fcc` を測定によってではなく構成によって劣位に固定していた。さらにベンダ推奨の `-Kfast,openmp` は 0.245 (反復数 5 での 1 回測定) と最も遅い部類であり、「ベンダコンパイラをベンダ推奨フラグで使う」が最悪の選択となる。許可リストは形ではなく挙動で危険を定義する。ハーネスの再定義 (`-I/-D/-include`)、数値ライブラリのリンク (`-l/-L/-Wl` および富士通 `-SSL2/-SCALAPACK`)、出力先の変更、ツールチェーンの差し替え (`-B/-specs/-fplugin`)、反復間に状態を持ち越すもの (`-fprofile-*`、`-Nfjprof`)、記号検査を無効化するもの (`lto`)、`OpenMP` を無効化するもの (`-fno-openmp`、`-Knoopenmp`) を拒否する。

選択は許すがバージョンは固定する。ハーネスは `cc` を `PATH` から解決するため、実際のコンパイラは `job` の `module` 環境に依存し、本機ではこれが固定量ではない。ログインノードの `cc` は `GCC 11.5.0`、計算ノードは `GCC 8.5.0` であり、ベンダの `entry module` を `load` して初めて `fcc` が可視になる。270 `run` を多数の `job` として時間をかけて投入する以上、一部の `job` だけ `module` 状態が異なればコンパイラは条件と交絡する。そこで各 `node report` に解決後のパスと `--version` を記録し、さらに期待バージョンを環境変数で宣言した場合は不一致をインフラ障害として停止する (候補は合法的なシステムを要求しており、別物を供給したのは環境だからである)。候補が選ぶ変動は探索軸、環境が押し付ける変動は交絡、という区別に基づく。ただしエージェントと評価器は同一 `uid` で動作し、`OS` レベルの隔離はない。この防御は改変検出と結果検証であり、悪意ある候補に対する完全な `containment` ではない。

j) モデルと実行環境: モデル `endpoint` は `cerebras/gpt-oss-120b` に固定し、`OpenAI` 互換 `LiteLLM client` で推論する。temperature は 0.7、context 長は 32768 token、client retry は 1 回とし、run seed を API へ渡す。top-*p* と最大 completion token は明示指定せず provider default を用いた。immutable な重み・serving revision、量子化、provider 側精度、`endpoint` が seed を deterministic に扱ったかは trace に残っていない。したがって API seed は best-effort の要求であり、同一 completion の保証ではない。1 課題・1 条件・1 seed の run を別々の Slurm array element として sbatch し、各 element の探索、ビルド、計測、採点を `fx700` 1 ノード (aarch64、48 物理コア、`OMP_NUM_THREADS=48`) 内で完結させる。計測対象の `OpenMP` は `OMP_PROC_BIND=spread`、`OMP_PLACES=cores`、`OMP_DYNAMIC=FALSE` に固定する。

ページ配置に関わるカーネル設定は測定して記録する。本機は NUMA 4 ノード構成で、NUMA 自動バランシングが有効である (`numa_balancing=1`、scan delay 1000 ms、scan period 1000–60000 ms)。すなわちカーネルはアクセス局所性に応じて実行中にページを移動させる。実際にこのノードの累積で 1.6×10^7 ページが移動している (`/proc/vmstat` の `numa_pages_migrated`)。透過的的巨大ページは無効 (never) であり、`zone_reclaim_mode=0` である。

この設定は stencil の測定不安定性の候補説明であったが、実測により否定した。同一ビルドを 14 回採点し、各採点の前後で `numa_pages_migrated` の増分を読むと、中央値付近の 12 回では 295 ページ移動しているのに対し、5% 以上外れた 2 回はいずれも 0 であった (比 0.00)。移動が原因なら外れた採点でこそ多いはずで、観測は逆である。カウンタはシステム全体であり他ジョブの寄与を含むため、この向きの結果は雑音では説明できない。なお本稿の他の測定でも、同一ビルドの外れ値は 338 ms と 486 ms のように複数の水準を取り、単一の二状態遷移でもない。他ジョブとの同居による干渉も原因ではない。計算ノードは排他確保であり (partition の `OverSubscribe=EXCLUSIVE`、各 job が 48 CPU 全数を確保)、1 ノードに 1 job しか載らない。機構は最終的に、採点に使うバイナリそのものを反復実行することで特定した (後述)。

まず本稿が検討し棄却した説明を列挙しておく。査読者が同じ経路を辿り直す必要がないようにするためである。(i) 特定ノードの不調——同一ノードで `gcc`、および `gemm/spmm` は 0.3–0.9% に収まる。(ii) 候補と参照の実行順序——候補先行 96 回の中央値 1.1342 に対し参照先行 72 回で 1.1386。(iii) NUMA 自動バランシングによるページ移動——上記のとおりの比 0.00。(iv) 他ジョブとの干渉——ノードは排他確保である。(v) カーネル計算そのもの——参照の確保・first touch・掃引だけを複製した独立プログラムを 20 プロセス走らせると全て 230.6–236.8 ms に収まり、外れ値は一度も再現しない。掃引 1 回は 0.92 ms で 240 回とも一定であり、最初の 10 回と最後の 10 回の比は 0.98 で劣化もない。確保を計測窓の内側に移して 30 プロセスで測り直しても、確保は 0.037–0.042 ms (全体の 0.02%) で、229.5–238.1 ms に収

まる。(vi) driver 側の入出力——後述の内部タイマがこれらを含まない。

機構——計測窓内の確保と *first touch*: 複製ではなく採点対象そのものを測ることで機構が定まった。採点に使う `stencil_main.c` と参照カーネルをハーネスと同じフラグで構築し、実際の `problem.bin` に対して反復実行して、ハーネスが採点に用いるのと同じ内部タイマ (`timing.bin`) を読む。この内部タイマは問題ファイルの読込・出力の書出し・プロセス起動を含まない。掃引回数を変えて各 20 プロセス測ると式 (1) となり、ハーネスに記録された $50.57 + 0.9298 n_t$ を有効数字 4 桁で再現する ($n_t = 1, 30, 120, 240$ の 4 点とも残差 0.03% 以下)。すなわち固定項 50.6 ms は入出力ではなく、計測窓の内側で参照が自身の 268 MB を確保し *first touch* する費用である。

分散はこの固定項に集中している。 $n_t = 1$ では掃引が測定の 1.8% にすぎないが、100 プロセスの範囲は 33.9–52.0 ms (中央値 51.2, $p_{95} |x - \text{med}|$ が 4.73%) と 34% に広がる。対して $n_t = 240$ では 60 プロセスが 270.7–279.0 ms (p_{95} 1.07%) に収まる。掃引 1 回あたりの費用は $n_t = 1$ から 240 まで 0.03% で再現するのだから、ぶれているのは掃引ではなく確保と *first touch* である。別の 20 プロセスの組では $n_t = 30$ で +38.3 ms、 $n_t = 240$ で +28.1 ms の遅い外れ値が出ており、超過は n_t に比例せず ms でほぼ一定である。これも出所が固定項であることを示す。本稿が当初この説明を棄却した根拠は「超過が掃引回数に対し 6.6 倍に増える」という採点水準の観測であった。採点対象バイナリの直接掃引はその増大を再現しない一方、ぶれる成分が確保と *first touch* であることは上のおり直接示されるので、棄却は撤回する。

振幅を採点時の水準まで押し上げる条件も特定した——ページキャッシュの冷えである。素のバイナリ、268 MB を常駐させた親からの起動、単発の採点呼出しの 3 条件を各 40 プロセス測っても最大の超過は +7.7% が 1 件にすぎず、いずれも再現しない。再現したのは 3 形状を通して 14 回連続採点したときだけで、そこでは中央値を 5% 超えた反復 31 件がすべて最初の 2 採点に集中し、3 回目以降は 0 件であった (最大 2086 ms、通常 214 ms)。単一形状の反復では 370 プロセス測っても一度も出ない。形状を切り替えて初めて、参照解 5.1 GB 全体を読むことになるからである。

予測を立てて検証した：採点前にこの 5.1 GB を読んでおけば、最初の 2 採点の外れ値は消えるはずである。2 s の事前読み込みを足した 14 回連続採点では、採点 1 が 2.1–5.3%、採点 2 が 1.3–1.8% に収まり、全 630 反復の範囲は 200.5–278.7 ms (事前読み込みなしでは 200.5–2085.8 ms) となった。形状別の採点間ばらつきは 78.4%/104.1% から 0.42%/0.33% へ下がる。

独立プログラム (v) が再現できなかった理由も同じ測定で説明がつく。そのプログラムの *first touch* は 7.1–10.3 ms であったのに対し、採点経路では同じ処理が 50.6 ms かかる。採点用 driver は `jacobi` を呼ぶ時点で既に `u0` と `u` の 268 MB を常駐させており、その上でカーネルがさらに 268 MB を確保する。確保と *first touch* の費用はプロセスの既存常駐量に依存するため、常駐の

ない複製では再現しない。複製による反証は、複製しなかった条件の分だけ弱い。

この結果は測定手順に反映した。配列の各要素は別ノードで走り、ページキャッシュはノード毎であるから、事前読み込みを入れなければ全 *run* が冷えた状態で最初の採点を行う。本稿の測定は各要素の冒頭で当該課題の参照解を読み込む (実測 2–19 s)。

したがって `stencil` だけが分解帯を満たさない理由は課題の構造にある。3 課題のうち `stencil` だけが、候補自身の 268 MB の確保と *first touch* を計測窓の内側に置く。 $n_t = 240$ という選択はこの固定項の比率を 64.4% ($n_t = 30$) から 18.5% へ下げるが、固定項そのものは消えない。参照解を計算する制御器側の BLAS は 1 thread に制限し、参照計算の worker が直後の候補計測と競合することを防ぐ。各 *run* 内の BFTS node は直列実行し、兄弟 node の自己テストと評価を重ねない。array の並行数は投入時の空きノード数から決め、条件数 9 の倍数に切り下げる (上限は空きノード数ではなく LLM endpoint の同時接続で決まる。実際に用いた値は `manifest` に記録する)。低い array index が先に割り当てられる可能性に備え、seed ごとに各条件が各 index 位置へ順に現れるよう配置を循環する。この循環により、並行して走るとの集合も各条件を同数含む——270 element の写像を全数検査し、各セルがちょうど 1 回現れること、各 index 位置で条件が均衡すること、並行数 9/18/27 のいずれでも各波が均衡することを確認した。並行数の上限はノード数ではなく LLM endpoint の同時接続で決まる：終端まで再試行を使い切った rate limit は node 予算を 1 消費し、node 水準の再試行は存在しない。候補と baseline は同一 allocation 内で対として測る。job ID、array index、node 名、CPU 数を `manifest` に保存し、条件別 node 割当も報告する。

各 array element は自ノードのページキャッシュへ当該課題の参照解を先に読み込む (実測 2–19 s)。ページキャッシュはノード毎であり、これを行わないと全 *run* が冷えた状態で最初の採点を行う (前掲のとおり外れ値はそこに集中する)。handoff 関連の外部環境変数は *run* 起動前に消去し、解決後の設定を記録する。

k) 解析方法と実装凍結: 解析単位は 1 課題・1 seed・1 条件の BFTS *run* である。各課題・各条件を 30 seed で別個に反復し、全体で $3 \times 3 \times 30 = 270$ *run* とする。主要対比は E-W と S-E の 2 つである。同一 seed は *matched block* とし、model API に同じ seed を渡し、`stencil` の探索入力を固定する。`gemm` と `sppm` の探索時 generator は固定設定を使い、全課題の最終再測定は block 内で同じ $s + 10^6$ を使う。seed は Slurm node を決めず、3 方針の実現した軌跡も共有しない。

主要推定量は、無効な最終再測定を 0 とした native speedup の seed 内算術平均差である。これは科学的失敗を含む all-run outcome を保つためであり、0 で未定義となり失敗を条件付けで除外する log ratio を主解析にしなかった。20,000 回の両側 sign-flip test は、効果 0 の帰無仮説下で *matched difference* の符号が対称・交換可能であることを仮定する。5,000 回の seed-pair percentile bootstrap 95%信頼区間も報告する。有効率は exact McNemar 検定と対応リスク差、両条件で有効かつ

正值の pair に限定した大きさは log-speedup 差と幾何平均比 (GMR) として分けて報告する。最終結果は全て有効だったが、GMR の「条件付き」という位置付けは事前の失敗規則による。3 課題×2 主要対比の permutation p 値へ Holm 補正を適用する。探索中の親子改善率、親破壊率、node 有効率は副次解析とし、親子率からは no-op 子を除外する。

科学的無効は candidate_invalid または measurement_invalid と記録し、0 を与える。

infrastructure_error は測定値ではない。非零終了と出力欠損も含めて解析から除外し、条件別件数を報告する。主要対比では、いずれか一方が infrastructure_error の seed block を除き、除外 seed を明示する。

ここで「凍結」は外部 preregistration ではなく実装凍結を指す。3 条件のスキーマ、プロンプト、ツール、モデル、 $N = 10$ 、ReAct 上限、seed 0–29、評価器、ハーネス、主要解析コードを 1 つの commit で固定し、最初の confirmatory root をその後に開始する。予備実験ではこれを commit cff60a7 (2026-07-29 21:16:49 JST) で行い、最初の root を 21:19:18 JST に開始した。本稿の条件では、評価器・ハーネス・解析コードのいずれも予備実験以降に変更している (分母、問題サイズ、計上時間、thread team 位置、公開ツール、および sign-flip 区間の追加) ため、凍結 commit は本稿の条件で最初の run を開始する時点のものとなる。run ごとに実験 driver、解析・batch script、制御コード、課題ハーネス、設定、git status、binary diff と各 hash を bundle 化した。先行 smoke run は実行可能性と収集経路の確認だけに用い、効果推定にも seed 数の決定にも用いていない。投入時に共通 source fingerprint を固定し、各 worker の開始前・終了後と集計時に照合した。不一致 run は解析対象外とした。

30 block は formal power calculation から決めておらず、事前に実用上最小の効果量または同等性幅も定めていない。したがって、非有意差を同等性の証拠とは解釈しない [21]。また、percentile bootstrap 区間と sign-flip 検定の見かけ上の不整合を解消するため、査読後の事後解析として、同じ算術平均対応差に対する 500,000 回 sign-flip 検定を反転した 95% 区間を追加する。凍結済みの p 値と Holm の検定族は変更しない。さらに 30 個の対応差をすべて図示する。

V. 結果

未取得。本節の測定値は、v2 として本稿に記述した条件が確定する前の予備実験 (270 run) で得られたものであり、その条件は本稿の実験設計と一致しない。相違は分母 (naive から competent 参照へ)、gemm の問題サイズ (512³、作業集合 6.0 MiB で L2 内)、stencil の掃引回数 ($n_t = 30$ 、計測窓の 64% が掃引以外)、計上時間の定義 (wall clamp 有)、thread team の生成位置 (計測窓内)、ReAct step 上限 (25)、公開ツール (6 個) に及ぶ。thread team の位置だけでも凍結参照の内部時間は gemm で 37.5–54.3%、spmm で 14.0–41.9% 変わり、stencil では 2 倍以上になるため、両者の speedup は数値として比較できない。したがって予備実験の測定値

表 VI
提案カテゴリの採用率と、提案を受けていない条件での基底率 (予備実験 CORPUS、EVIDENCE ONLY/EVIDENCE+SELF-REPORT の親子 819 組、EVIDENCE ONLY 810 組、WORKSPACE ONLY 819 組)。

カテゴリ	提案回数	採用率	基底率 Evidence only	基底率 Workspace
ブロッキング	1180	59.5%	50.0%	
ベクトル化	635	41.3%	37.9%	
フラグ	221	35.7%	29.4%	
prefetch	164	29.9%	11.0%	
スケジューリング	335	15.8%	23.0%	
first touch	206	12.1%	7.4%	
アンロール	82	6.1%	1.6%	

は破棄し、本節は本稿の条件で再取得したデータで置き換える。旧本文は以下にコメントとして残す。

VI. 考察

結果に依存する記述は保留。本節のうち、条件間比較の帰結を述べる部分は、破棄した予備実験の測定値に基づいていた。本稿の条件によるデータ取得後に記述する。以下には、測定結果によらず本稿の設計から定まる部分だけを残す。旧本文はコメントとして保存する。

a) 本実験が支持しうる範囲: 本設計は同等性幅を事前に定めていないため、条件差が有意でない場合も「Self-Report は既定値にすべきでない」という一般的判断は導けない。逆に有意差が出た場合も、それは本稿の bundle・model・課題における効果であり、handoff 一般の性質ではない。採用判断には、許容する token・金額・待ち時間・失敗率と、最小限必要な改善率を事前に定める必要がある。本稿の設計から得られる手続き上の含意は、Evidence と Self-Report を別 channel に保ち、後者を測定可能な介入として扱うことに限られる。

b) Self-Report は探索経路をどう変えたか: 最終スコアだけでは「Self-Report が効いたか」に答えられず、機序を伴わない主張になる。そこで予備実験の corpus に対し、親の next_steps_hints を最適化カテゴリへ分類し、親子のソース差分を同じカテゴリへ分類し、提案カテゴリが子の差分に現れた場合を採用と数えた。これは意図の読み取りではなく代理指標なので、提案を一切受け取っていない他 2 条件で同じカテゴリが親子間で変化する割合、すなわち基底率を併記する。基底率なしでは、最適化エージェントが元来行う操作を採用と誤読する。

表 VI に結果を示す。基底率を明確に上回るのは prefetch だけであり (29.9% 対 11.0%/7.6%)、ブロッキングとベクトル化は数ポイントの上乗せにとどまる。スケジューリングは 335 回提案されながら採用率 15.8% で、提案を受けていない Evidence only の基底率 23.0% を下回る。first touch は 12.1% で基底率 7.4%/3.7% を上回る。したがって「Self-Report が探索方向を決める」という描像は、prefetch と first touch では弱く支持され、スケジューリングでは支持されない。分類の粒度でしか言えない主張であることを強調する。

この表の first touch 行は当初 0% と報告していた。分類器が #pragma omp parallel for の直後の 1 行にのみ書き込みを探しており、宣言や開始波括弧を挟む通常の書き方を全て見落としていたためである。凍結参照自身の touch ループすら検出できておらず、それに気づいたのは同じ分類器を参照実装に当てたときであった。窓を数行に広げて再解析した値が表の値である。採用の有無で子スコアの対親変化を比べても中央値 -0.72% 対 -0.33% であり、採用側が優位ではない。

この解析は予備実験の corpus に対するものであり、本稿の条件で再取得したデータに対して同じ手続きを適用する。カテゴリ分類は正規表現による代理指標であって意図の読み取りではないため、分類器と全対を成果物として公開する。

c) 同定できない機序: Evidence+Self-Report は意味内容と文脈長を同時に増やす。長さを揃えた placebo、shuffled Self-Report、短縮 Self-Report がないため、有益な内容が注意分散で相殺された場合、提案品質が低い場合、Evidence の言い換えに留まる場合を分離できない。事後監査は不適合提案と明示的不整合を数えるが、Self-Report 品質別の因果効果は推定しない。探索方向の固定化も現在の指標では直接測定していない。したがって本稿の結果は、害または利益の機序ではなく、明示的な bundled policy の end-to-end 比較として解釈する必要がある。

VII. 限界

- 1) 本稿の測定値自体が、環境記録より前に測られている。第 ?? 節は、測定条件を記録せずに得た比率は運用設定の関数だと主張する。本稿が報告する band、実測天井、ツールチェーン表、5.9 倍、1.313 倍、6.8 倍は、その記録機構を実装する前に測ったものであり、条件は digest ではなく作業記録から分かるにすぎない。既知の依存軸（コンパイラ、-Nclang の有無、ラージページ方針）については本文中に条件を明記したが、当時気づいていなかった変数について同じ保証はない。以後の測定は digest を伴う。
- 2) Workspace only でも親コード中のエージェント記述コメントはそのまま継承される（除去処理を置いていない）。したがって自然言語を除去した対照ではなく、明示的 report channel を除いた対照である。
- 3) Self-Report の内容と入力長を分離していない。placebo、shuffled、短縮条件が必要である。
- 4) 同一 uid で動く候補と評価器の間に syscall・filesystem containment がなく、静的検査と hash 検査だけでは悪意ある改変を完全には防げない。とくに計上時間は候補プロセス内の計測に依存し、これを外側から数値的に反証する機構は置いていない。
- 5) 非有意差は効果ゼロの証明ではない。同等性幅を事前指定していない。
- 6) 探索予算は node 数 10 に固定しており、予算を変えたときの感度を測っていない。handoff の効果が「同じ予算でより良い解に届く」ものか「同じ解に

より少ない node で届く」ものかを本稿は区別できない。予算を 10 から 20 へ動かした比較は、1 条件あたり 30 seed × 3 課題を再度要するため本稿の範囲外とした。実測では確定構成 1 run が gemm 4.8 分、spmm 19.5 分、stencil 51.4 分であり、予算を倍にした比較は 114 node 時間の追加を意味する。

- 7) handoff は毎 node 常時発火する固定 policy であり、いつ渡すかを学習・選択する機構は置いていない。親が渡す内容をスキルとして蓄積・再利用する構成（過去 node から再利用可能な手順を抽出し、後続に提示する）は、本稿の 3 条件のいずれでもない。これは handoff 経路の内容を変える設計であって有無を変える設計ではないため、本稿の対照（Workspace only/Evidence only/Evidence+Self-Report）では分離できない。
- 8) 単一 model endpoint、sampling 設定、A64FX、3 kernel、10 node に限られる。N = 5, 20、expert-tuned baseline、実アプリケーションは新しい実験を要し、既存 trace から復元できない。
- 9) immutable model revision、provider 精度、価格、latency、SpMM の最終 nnz、hardware counter を記録していない。
- 10) 親子改善・破壊率は探索木内で依存し、探索方向の固定化を直接測る指標ではない。
- 11) 性能測定には残留ノイズがあり、選択的または外部検証付き Self-Report も評価していない。
- 12) stencil の測定は絶対時間では大きく揺れる。個々の反復は 214ms のところ 2086ms まで飛ぶことがある。原因はページキャッシュの冷えであり、参照解 5.1 GB を読み終えるまで親の入出力が子の計測窓を止める（外れ値 31 件がすべて最初の 2 採点に集中し、事前に 5.1 GB を読む 2s を足すだけで消える）。比では相殺するため score への影響は小さいが、絶対時間で報告する量には効く。また各配列要素は別ノードで走りページキャッシュはノード毎であるため、事前読み込みを入れない限り全 run が冷えた状態で最初の採点を行う。本稿の測定はこの事前読み込みを行う。
- 13) 反復数 15 の測定 band は 3 課題いずれも目標未達である（gemm 1.06 倍、spmm 1.005 倍、stencil 1.40 倍。表 IV）。すなわち予備実験と同じ兄弟間差の分布を仮定する限り、探索は band 未達の差にある兄弟を分解できていない。主要対比は 30 seed block の対応差に対する検定であるためこの限界は主結論より探索経路の解釈に効くが、兄弟選択が偶然を含むという前提で読む必要がある。
- 14) ベンダコンパイラの可用性は job の module 環境に依存する。全 run で同一の module 状態を保たなければ、選択肢の有無が条件と交絡しうる。ハーネスは選択できなかった事実を記録して既定へ退避するため取り違えは起きないが、可用性そのものは投入側の責任である。
- 15) Self-Report を評価後に生成する設計は、内省が評価値をそのまま繰り返せば Evidence+Self-Report を Evidence only の言い換えに縮退させる。prompt

で測定値の再掲を禁じ、提案の採用率を無提案条件の基底率と対比して事後に測るが、言い換えの度合いを直接定量する指標は置いていない。

- 16) 凍結参照は採点する問題サイズ向けに調整されており、他のサイズでの性能を代表しない。たとえば 2048 立方では蓄積ブロックが L1d 全体を占め、ブロッキングを外した実装に 4.6 倍劣る。本稿のスコアは採点ケースにおける比としてのみ解釈できる。

VIII. 結論

親コード継承下の handoff を Workspace only、Evidence only、Evidence+Self-Report に分解し、3 HPC カーネル上で比較する実験設計を示した。本稿の寄与は、この分解自体と、それを測定可能にするための評価系にある。すなわち、naive ではなく competent な凍結参照を分母に置くこと、作業集合が最終段キャッシュに収まらない問題サイズを選ぶこと、計上時間を候補の内部タイマに一本化し thread team 生成を計測窓の外に出すこと、そして偽装を wall 下界ではなく構造的制約で防ぐことである。これらは予備実験で条件差と同程度以上の系統差を生じることが実測されたため、比較可能性の前提条件として設計に組み込んだ。

評価系については、設計から演繹するのではなく実測して初めて分かった要件が三つあった。第 1 に、分母が competent であることは宣言ではなく検査を要する。当初の gemm 参照は 3 形状のうち 1 つで教科書どおりの実装に 1.313 倍劣っており、エージェントが実際に生成した候補と突き合わせて初めて判明した。第 2 に、採点する問題サイズは規模とキャッシュ収まりだけでは決まらない。 $n = 500$ は 48 スレッドに対し 10.4 行/スレッドであり、候補が書くであろう行並列実装がその形状では安定に測れない。形状を $1500 \times 600 \times 1500$ に替えて偏りと分散が同時に解消した。第 3 に、ツールチェーンは探索空間の一部であって固定すべき定数ではない。同一の fcc が -Nclang の有無で 6.8 倍動き、その差はフラグ許可リストの書き方だけで塞がれていた。いずれも「測定できているか」を測る手続きを置かなければ気づけない種類の欠陥であり、本稿はその手続きを成果物として公開する。

条件間比較の結果は、本稿の条件で再取得したデータにより記述する。得られる結論は、Self-Report 品質のばらつき、意味内容と文脈長の交絡、Workspace only に残る source comment channel を含む、明示的 always-on handoff policy の比較として解釈されるべきものである。さらに次の実験では、長さを揃えた対照、事前の実用閾値、探索予算感度、実アプリケーションを導入する必要がある。

DATA AVAILABILITY

本実験システムは、公開リポジトリ ARI v0.9.1 (<https://github.com/kotama7/ARI.git>) を基盤とする拡張である。実験・解析実装は、次の private リポジトリに保存している (閲覧には権限が必要)。

<https://github.com/kotama7/Agentic-HPC-handoff-based-on-ARI>

予備実験には commit cff60a7 を用いた。本稿の条件で用いる commit は、評価器・ハーネス・解析コードを本稿の

記述どおりに変更したうえで、最初の run を開始する時点で確定する。これとは別に予備実験の再解析・監査に必要な記録一式を収めた「270-run Agentic HPC handoff 実験の再現・監査用データセット」を、容量のため Git 追跡対象外の archive とした (ローカルファイル名は workspace.zip)。同データセットには、課題ハーネス、Slurm 投入・解析スクリプト、各 run/node の候補コードと評価記録、LLM の prompt/cost trace、最終独立再測定、checkpoint、AArch64 実行環境が含まれる。本稿の条件で実測した評価系の検証 (参照実装の対 corpus 比較、team 生成位置、プロセス overhead、エイリアシング、キャッシュ階層) は、いずれも測定スクリプトと生の出力を checkpoint として同 archive に含める。このデータパッケージは Zenodo で公開する予定である (プレースホルダー URL : <https://zenodo.org/records/PLACEHOLDER>)。投稿前に private repository へ artifact 査読者のアクセスを付与し、上記 placeholder を確定 DOI に置換する必要がある。現時点ではこの 2 点は提出上の未完了事項である。実行環境、hash 照合、再解析コマンド、期待出力、縮小確認手順は同梱の Artifact Description に記載する。

参考文献

- [1] Z. Jiang, D. Schmidt, D. Srikanth, et al. (Weco AI). AIDE: AI-Driven Exploration in the Space of Code. *arXiv:2502.13138*, 2025.
- [2] J. Li, G. Li, X. Zhang, et al. CodeTree: Agent-guided tree search for code generation with large language models. *NAACL* (arXiv:2411.04329), 2025.
- [3] N. Dainese, M. Merler, M. Alakuijala, P. Marttinen. Generating code world models with reinforcement learning and Monte Carlo Tree Search (GIF-MCTS). *NeurIPS* (arXiv:2405.15383), 2024.
- [4] A. Ouyang, S. Guo, S. Arora, et al. KernelBench: Can LLMs write efficient GPU kernels? *arXiv:2502.10517*, 2025.
- [5] S. Yao, J. Zhao, D. Yu, et al. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.
- [6] N. Shinn, F. Cassano, A. Gopinath, et al. Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 2023.
- [7] S. Yao, D. Yu, J. Zhao, et al. Tree of Thoughts: Deliberate problem solving with large language models. *NeurIPS*, 2023.
- [8] A. Zhou, K. Yan, M. Shlapentokh-Rothman, et al. Language Agent Tree Search unifies reasoning, acting, and planning in language models. *ICML* (arXiv:2310.04406), 2024.
- [9] R. C. Whaley, A. Petitet, J. J. Dongarra. Automated empirical optimizations of software and the ATLAS project. *Parallel Computing*, 27(1–2):3–35, 2001.
- [10] M. Frigo, S. G. Johnson. The design and implementation of FFTW3. *Proceedings of the IEEE*, 93(2):216–231, 2005.
- [11] J. Ansel, S. Kamil, K. Veeramachaneni, et al. OpenTuner: An extensible framework for program autotuning. *PACT*, 303–316, 2014.
- [12] T. Chen, T. Moreau, Z. Jiang, et al. TVM: An automated end-to-end optimizing compiler for deep learning. *OSDI*, 578–594, 2018.
- [13] H. C. Edwards, C. R. Trott, D. Sunderland. Kokkos: Enabling manycore performance portability through polymorphic memory access patterns. *J. Parallel Distrib. Comput.*, 74(12):3202–3216, 2014.
- [14] B. Cui, T. Ramesh, O. Hernandez, K. Zhou. Do Large Language Models Understand Performance Optimization? *arXiv:2503.13772*, 2025.
- [15] A. Rahman, V. Cvetkovic, K. Reece, et al. MARCO: A Multi-Agent System for Optimizing HPC Code Generation Using Large Language Models. *arXiv:2505.03906*, 2025.
- [16] S.-i. Hayashi, K. Morita, D. Mukunoki, T. Hoshino, T. Katagiri. VibeCodeHPC: An Agent-Based Iterative Prompting Auto-Tuner for HPC Code Generation Using LLMs. *arXiv:2510.00031v3*, 2026.
- [17] Z. Li, J. Ren, Z. Tang, J. Zheng, Z. Wang. AutoPass: Evidence-Guided LLM Agents for Compiler Performance Tuning. *arXiv:2606.20373*, 2026.
- [18] B. Mikek, D. Vashchilenko, B. Lu, P. Xu. Agentic Code Optimization via Compiler-LLM Cooperation. *arXiv:2604.04238*, 2026.
- [19] H. Jing, et al. PerfCodeBench: Benchmarking LLMs for System-Level High-Performance Code Optimization. *arXiv:2605.15222v2*, 2026.
- [20] R. Mikasa, S.-i. Hayashi, D. Mukunoki, T. Hoshino, T. Katagiri. Improving HPC Code Generation Capability of LLMs via Online Reinforcement Learning with Real-Machine Benchmark Rewards. *arXiv:2602.12049*, 2026.

- [21] D. Lakens. Equivalence Tests: A Practical Primer for t Tests, Correlations, and Meta-Analyses. *Social Psychological and Personality Science*, 8(4):355–362, 2017.